

NOVACART

SQL Hints

Guidance without complete solutions

Platform: Microsoft SQL Server / SSMS

Use this file only when you are stuck.

The hints are intentionally designed to help you think through the problem without giving you the final query.

Hint 1 — Think: identify what the question is really asking.

Hint 2 — Concept: choose the SQL idea that fits the requirement.

Hint 3 — Structure: use the provided SQL skeleton as a starting point, then complete it yourself.

How to Use This Hint Book

Do not read all three hints immediately. Start with Hint 1. If you can solve the question, stop there. If you are still stuck, move to Hint 2, then Hint 3.

The Hint 3 sections may show table names, clauses, functions, or a partial query structure. They intentionally do not provide a complete solution.

Hint numbering uses the format M1-Q1: Mission 1, Question 1. This matches the order of the Student Assignment.

Important SQL Server note: use SQL Server syntax and functions appropriate for SSMS.

Mission 1 — Customer & Product Overview

M1-Q1 — Display all customers.

Hint 1 — Think

Which table is responsible for storing customer information?

Hint 2 — Concept

Use a basic SELECT query against the Customers table.

Hint 3 — Structure

Start with:

SELECT ...

FROM ...;

M1-Q2 — Display product name, category, and current price.

Hint 1 — Think

You do not need every column in the Products table. Identify the three requested attributes.

Hint 2 — Concept

Use SELECT to choose specific columns from Products.

Hint 3 — Structure

Start with:

SELECT column1, column2, column3

FROM Products;

M1-Q3 — Products whose current price is greater than 5000.

Hint 1 — Think

This is a row-filtering problem. What condition separates the required products?

Hint 2 — Concept

Use WHERE with a comparison operator.

Hint 3 — Structure

Start with:

SELECT ...

FROM Products

WHERE price > ...;

M1-Q4 — Customers ordered by join date, newest first.

Hint 1 — Think

The rows should all remain; only their order changes.

Hint 2 — Concept

Use ORDER BY. Newest dates come first, so think about descending order.

Hint 3 — Structure
Start with:
SELECT ...
FROM Customers
ORDER BY join_date ...;

M1-Q5 — Total number of customers.

Hint 1 — Think
You need one number representing how many customer records exist.

Hint 2 — Concept
COUNT is designed to count rows or values.

Hint 3 — Structure
Start with:
SELECT COUNT(...)
FROM Customers;

Mission 2 — Aggregation & Business Totals

M2-Q1 — Average current product price.

Hint 1 — Think
You need one summary value from all current product prices.

Hint 2 — Concept
Use AVG on the price column.

Hint 3 — Structure
Start with:
SELECT AVG(...)
FROM Products;

M2-Q2 — Highest and lowest current product prices.

Hint 1 — Think
Think about the two opposite extremes in a numeric column.

Hint 2 — Concept
Use MAX and MIN together.

Hint 3 — Structure
Start with:
SELECT MAX(...), MIN(...)
FROM Products;

M2-Q3 — Total available stock quantity.

Hint 1 — Think
Stock quantity is stored per product. You need the combined amount.

Hint 2 — Concept
Use SUM on stock_quantity.

Hint 3 — Structure
Start with:
SELECT SUM(...)
FROM Products;

M2-Q4 — Total amount recorded in Payments.

Hint 1 — Think

Payment amounts are stored in one table. You need their overall total.

Hint 2 — Concept

Use SUM on Payments.amount.

Hint 3 — Structure

Start with:

```
SELECT SUM(...)
```

```
FROM Payments;
```

M2-Q5 — Number of orders for each order status.

Hint 1 — Think

This is not one total. You need one result per status.

Hint 2 — Concept

Combine COUNT with GROUP BY status.

Hint 3 — Structure

Start with:

```
SELECT status, COUNT(...)
```

```
FROM Orders
```

```
GROUP BY ...;
```

M2-Q6 — Total payment amount for each payment method.

Hint 1 — Think

Think of method as the category and amount as the value to summarize.

Hint 2 — Concept

Group Payments by method and sum amount.

Hint 3 — Structure

Start with:

```
SELECT method, SUM(...)
```

```
FROM Payments
```

```
GROUP BY ...;
```

Mission 3 — Order & Sales Analysis

M3-Q1 — Total sales amount for each order.

Hint 1 — Think

One order can contain multiple order-detail rows. What should happen to those rows?

Hint 2 — Concept

Calculate each line using quantity × historical unit price, then aggregate by order.

Hint 3 — Structure

Start with:

```
SELECT order_id, SUM(... * ...)
FROM OrderDetails
GROUP BY ...;
```

M3-Q2 — Orders whose total sales exceed 5000.

Hint 1 — Think

The condition is applied after calculating a total for each order.

Hint 2 — Concept

An aggregate result is filtered with HAVING, not WHERE.

Hint 3 — Structure

Start with:

```
SELECT order_id, SUM(... * ...)
FROM OrderDetails
GROUP BY order_id
HAVING SUM(... * ...) > ...;
```

M3-Q3 — Each order with the customer's name, date, and status.

Hint 1 — Think

Orders contain customer_id, but the customer's name is stored in another table.

Hint 2 — Concept

Join Orders to Customers using their matching key columns.

Hint 3 — Structure

Start with:

```
SELECT ...
FROM Orders o
JOIN Customers c
ON ...;
```

M3-Q4 — Each order with its products, quantities, and historical unit prices.

Hint 1 — Think

An order does not directly store the product name. Follow the relationship through the order-detail table.

Hint 2 — Concept

You need Orders → OrderDetails → Products.

Hint 3 — Structure

Start with:

```
FROM Orders o
JOIN OrderDetails od ON ...
JOIN Products p ON ...
```

M3-Q5 — Total amount spent by each customer.

Hint 1 — Think

Spending is stored as payment amounts, while the customer is linked through Orders.

Hint 2 — Concept

Join Customers to Orders, then Payments, and aggregate payment amount per customer.

Hint 3 — Structure

Start with:

```
SELECT ..., SUM(...)
FROM Customers c
JOIN Orders o ON ...
JOIN Payments p ON ...
GROUP BY ...;
```

Mission 4 — Reviews & Relationships

M4-Q1 — Number of reviews received by each product, including products with no reviews.

Hint 1 — Think

Ask yourself: should a product disappear if it has zero matching reviews?

Hint 2 — Concept

Use a LEFT JOIN from Products to Reviews, then count the review identifier.

Hint 3 — Structure

Start with:

```
FROM Products p
LEFT JOIN Reviews r ON ...
GROUP BY ...
```

M4-Q2 — All reviews with customer name and product name.

Hint 1 — Think

The review contains IDs, not necessarily the readable names.

Hint 2 — Concept

Join Reviews to Customers and Products.

Hint 3 — Structure

Start with:

```
FROM Reviews r
JOIN Customers c ON ...
JOIN Products p ON ...
```

M4-Q3 — Customers who have placed at least one order.

Hint 1 — Think

You need customers for whom a matching order exists.

Hint 2 — Concept

An existence test can be expressed with IN, EXISTS, or a suitable join. This mission is intended to practice subquery thinking.

Hint 3 — Structure

One possible structure is:

```
SELECT ...
FROM Customers
WHERE customer_id IN ( SELECT ... );
```

M4-Q4 — Products that have never been ordered.

Hint 1 — Think

Think about products whose `product_id` does not appear in `OrderDetails`.

Hint 2 — Concept

A `NOT IN` or `NOT EXISTS` approach can express the requirement. Be aware of `NULL` behavior when choosing `NOT IN`.

Hint 3 — Structure

Start with:

```
SELECT ...
```

```
FROM Products
```

```
WHERE product_id NOT IN ( SELECT ... );
```

M4-Q5 — Products that have never received a review.

Hint 1 — Think

Look for products whose `product_id` has no matching row in `Reviews`.

Hint 2 — Concept

Use an anti-match idea such as `NOT EXISTS` or `NOT IN`.

Hint 3 — Structure

Start with:

```
WHERE product_id NOT IN ( SELECT product_id FROM Reviews ... );
```

M4-Q6 — All customers and their number of orders, including customers with no orders.

Hint 1 — Think

The word including is the key. Customers with zero matching orders must remain.

Hint 2 — Concept

Start from `Customers` and use a `LEFT JOIN` to `Orders`. Count the order identifier rather than counting every joined row.

Hint 3 — Structure

Start with:

```
FROM Customers c
```

```
LEFT JOIN Orders o ON ...
```

```
GROUP BY ...
```

Mission 5 — Subqueries

M5-Q1 — Customers who placed more orders than the average number of orders among customers who placed at least one order.

Hint 1 — Think

First determine each customer's order count. Then determine the average of those counts. Finally compare each customer's count with that average.

Hint 2 — Concept

This requires two levels of aggregation and a comparison against a subquery result.

Hint 3 — Structure

Think in this shape:

```
SELECT customer..., COUNT(...)
FROM ...
GROUP BY ...
HAVING COUNT(...) > (SELECT AVG(...) FROM (...));
```

M5-Q2 — Products whose current price is above the average current product price.

Hint 1 — Think

The comparison value is calculated from the same Products table.

Hint 2 — Concept

Use a scalar subquery that returns the average price.

Hint 3 — Structure

Start with:

```
SELECT ...
FROM Products
WHERE price > (SELECT AVG(...) FROM Products);
```

M5-Q3 - Customers whose total spending exceeds the average total spending among customers who made at least one payment.

Hint 1 · Think

First calculate spending per customer among customers who made at least one payment. Then compare each total with the average of those customer totals.

Hint 2 · Concept

Use two levels of aggregation: calculate customer totals, then calculate the average of those totals and compare against it.

Hint 3 · Structure

Think about:
GROUP BY customer
HAVING SUM(...) > (SELECT AVG(...) FROM (...));

Mission 6 — Common Table Expressions

M6-Q1 — Using a CTE, calculate total revenue by month.

Hint 1 — Think

Revenue comes from Payments in this lab. The challenge is grouping payment dates into months.

Hint 2 — Concept

Build a CTE that groups Payments by a month expression and sums amount, then select from the CTE.

Hint 3 — Structure

Start with:

```
WITH MonthlyRevenue AS
(
```

```
SELECT ...  
FROM Payments  
GROUP BY ...  
)  
SELECT ...  
FROM MonthlyRevenue;
```

M6-Q2 — Using a CTE, calculate customer spending and return only customers above 10000.

Hint 1 — Think
Separate the calculation from the filtering step.

Hint 2 — Concept
First CTE: customer totals. Outer query: filter the CTE result.

Hint 3 — Structure
Start with:
WITH CustomerSpending AS
(
SELECT ... SUM(...) ...
FROM ...
GROUP BY ...
)
SELECT ...
FROM CustomerSpending
WHERE total_spent > ...;

Mission 7 — Window Functions

M7-Q1 — Rank customers by total spending using RANK().

Hint 1 — Think
You need total spending per customer before ranking those totals.

Hint 2 — Concept
Use GROUP BY to calculate spending, then RANK() OVER(ORDER BY the spending expression DESC).

Hint 3 — Structure
Start with:
RANK() OVER (ORDER BY SUM(...) DESC)
...

M7-Q2 — Rank products by total quantity sold using DENSE_RANK().

Hint 1 — Think
First total the quantity sold for each product.

Hint 2 — Concept
Use DENSE_RANK over the aggregated quantity in descending order.

Hint 3 — Structure
Start with:
DENSE_RANK() OVER (ORDER BY SUM(...) DESC)
...

M7-Q3 — Previous payment amount using LAG().

Hint 1 — Think
Previous means you need an ordered sequence of payments.

Hint 2 — Concept

Use LAG(amount) with an ORDER BY that is deterministic. The assignment asks for payment date and payment ID ordering.

Hint 3 — Structure

Start with:

```
LAG(amount) OVER (ORDER BY payment_date, payment_id)
```

...

M7-Q4 — Running total of payment amounts.

Hint 1 — Think

A running total is a windowed SUM over an ordered sequence.

Hint 2 — Concept

Use SUM(amount) OVER with ORDER BY payment_date, payment_id.

Hint 3 — Structure

Start with:

```
SUM(amount) OVER (ORDER BY payment_date, payment_id)
```

...

Mission 8 — Views

M8-Q1 — Create vw_revenue_by_month.

Hint 1 — Think

The view should store a reusable monthly revenue query.

Hint 2 — Concept

Use CREATE VIEW followed by the monthly revenue SELECT. The monthly grouping idea is the same as the CTE mission.

Hint 3 — Structure

Start with:

```
CREATE VIEW vw_revenue_by_month AS
SELECT ...
FROM Payments
GROUP BY ...;
```

M8-Q2 — Create vw_best_selling_products.

Hint 1 — Think

You need three pieces of information: product name, total quantity sold, and total revenue.

Hint 2 — Concept

Join Products to OrderDetails, group by product, and calculate both SUM(quantity) and SUM(quantity × unit_price).

Hint 3 — Structure

Start with:

```
CREATE VIEW vw_best_selling_products AS
SELECT p.name, SUM(...), SUM(... * ...)
FROM Products p
JOIN OrderDetails od ON ...
GROUP BY ...;
```

M8-Q3 — Create vw_customer_summary.

Hint 1 — Think

Your view must include every customer, even a customer with no orders.

Hint 2 — Concept

Use Customers as the starting table and LEFT JOIN Orders and Payments. Count distinct orders and handle missing payment totals appropriately.

Hint 3 — Structure

Start with:

```
CREATE VIEW vw_customer_summary AS
SELECT ... COUNT(DISTINCT ...), ...
FROM Customers c
LEFT JOIN Orders o ON ...
LEFT JOIN Payments p ON ...
GROUP BY ...;
```

Final Help — Common SQL Mistakes

If a query is not working, check these before asking for help:

- Did you use the correct table and column names?
- Is your JOIN condition connecting primary keys to the correct foreign keys?
- Did you accidentally use an INNER JOIN where the question requires unmatched rows to remain?

- If you use an aggregate function with normal columns, did you GROUP BY the appropriate columns?
- Are you filtering before aggregation with WHERE, or after aggregation with HAVING?
- Could a JOIN be creating duplicate rows and inflating your SUM or COUNT?
- For questions involving 'never', did you consider NOT EXISTS, NOT IN, or LEFT JOIN ... IS NULL?
- For window functions, is your ORDER BY deterministic when two rows have the same date or value?
- Are you using OrderDetails.unit_price for historical order revenue rather than the current Products.price?
- Did you test the query on your own data rather than assuming the sample data from another solution?

End of Hint Book

Try to solve the problem yourself before opening the next hint level.